

PROJECT REQUIREMENTS DOCUMENT

EDS 6352 – Natural Language Processing | Fall 2026

Classifying Gas Transmission & Gathering Pipeline Incident Causes

Using BiGRU · BiLSTM+Attention · BERT-base

Team 1 | Sakina Saifuddin · Muhammad Murtaza · Parva Shah

Generated: September 30, 2026

1. Document Purpose & Scope

This Product Requirements Document (PRD) serves as the authoritative reference for Team 1's EDS 6352 NLP Final Project. It translates the approved project proposal into granular, actionable engineering tasks, clearly assigns ownership across all three team members, specifies technical requirements for each model and experiment, and defines shared responsibilities such as preprocessing, evaluation, and deployment. All team members are expected to read and follow this document throughout the project lifecycle.

Course	EDS 6352 – Natural Language Processing, Fall 2026
Team	Team 1
Members	Parva Shah · Muhammad Murtaza · Sakina Saifuddin
Task Type	Multi-class Text Classification (8 classes)
Primary Input	NARRATIVE field — free-text incident description (avg ~1,019 chars)
Gold Label	CAUSE field — 8 PHMSA incident cause categories
Dataset	PHMSA_Gas_Transmission_Gathering_2002_Present.csv — 3,027 records (3,026 usable)

2. Dataset Overview

Source: U.S. DOT PHMSA – Gas Transmission and Gathering Incident Data (2002–2026).

The dataset was produced by merging the 2002–2009 historical PHMSA records with the 2010–present records, standardising all cause labels to the current eight categories. Only the six fields listed below are used in this project.

#	Cause Category (CAUSE)	Count	% of Total
1	EQUIPMENT FAILURE	782	25.8 %
2	CORROSION FAILURE	653	21.6 %
3	MATERIAL FAILURE OF PIPE OR WELD	367	12.1 %
4	EXCAVATION DAMAGE	360	11.9 %
5	NATURAL FORCE DAMAGE	301	9.9 %
6	OTHER OUTSIDE FORCE DAMAGE	208	6.9 %

7	OTHER INCIDENT CAUSE	192	6.3 %
8	INCORRECT OPERATION	164	5.4 %
	TOTAL (usable)	3,026	100 %

■ *Class Imbalance Note: EQUIPMENT FAILURE (782) is ~4.8× more frequent than INCORRECT OPERATION (164). Both experiments (class-weighted loss and smart oversampling) directly address this imbalance.*

3. Shared Preprocessing Pipeline

All three team members share equal responsibility for the preprocessing pipeline. The pipeline must be implemented in a single reusable script/notebook so that every model trains and evaluates on identical cleaned data.

3.1 Raw Data Cleaning

- Load **PHMSA_Gas_Transmission_Gathering_2002_Present.csv** — 3,027 rows × 6 columns.
- Drop the single row where NARRATIVE is null (report 20030038, year 2003) → **3,026 usable rows**.
- Convert all NARRATIVE text to **lowercase** — critical for FastText and BERT-base-uncased compatibility.
- Strip leading/trailing whitespace and normalise internal whitespace (collapse multiple spaces/newlines).
- Remove any residual HTML tags or encoded characters (e.g., &,
) found in older records.
- Encode the CAUSE label column as an integer class index (0–7) using a fixed, documented mapping.

3.2 Tokenisation Strategy (per model)

Model	Tokeniser	Vocabulary	Max Sequence Length
BiGRU (Parva)	Whitespace + word-level	FastText vocab (~300-d)	300 tokens (pad/truncate)
BiLSTM+Attn (Muhammad)	Whitespace + word-level	FastText vocab (~300-d)	300 tokens (pad/truncate)
BERT (Sakina)	BERT WordPiece tokeniser	BERT vocab (30,522 tokens)	512 tokens (BERT max)

3.3 Train / Validation / Test Split

- **Strategy: Stratified split** — preserves the 8-class distribution proportionally in every subset.
- **Ratios: 70% Training (2,118 rows) / 15% Validation (454 rows) / 15% Test (454 rows).**
- Use `sklearn.model_selection.train_test_split` with `stratify=y` and a fixed `random_state=42`.
- All three models **must use the identical split**. Save the indices to a shared file (e.g., `split_indices.json`) so any teammate can reproduce the same split.
- Validation set is used exclusively for hyperparameter tuning and early stopping.
- Test set is touched **only once** — for final evaluation after model training is complete.

4. Experiment Design — Class Imbalance Strategies

Every model must be evaluated under **two separate experiments**. Results must be reported for each experiment independently so the two strategies can be directly compared.

	Experiment 1	Experiment 2
Name	Class-Weighted Loss	Smart Oversampling
Training Data	Original training set (70% split, no augmentation)	Training set + AI-generated synthetic narratives for minority classes

Loss Function	Cross-entropy with per-class weights (inversely proportional to class frequency)	Standard unweighted cross-entropy loss
Oversampling	None — class weights compensate without altering data	Select real minority-class narratives → prompt an LLM to paraphrase/generate semantically equivalent examples → add to training set only
Val / Test Sets	Unchanged — identical to original split	Unchanged — identical to original split

Smart Oversampling Protocol: Select the 5–10 most representative training narratives from each under-represented class (INCORRECT OPERATION, OTHER INCIDENT CAUSE, OTHER OUTSIDE FORCE DAMAGE). Prompt an LLM (e.g., GPT-4o or Gemini) to generate 10–15 paraphrased variants per seed narrative, preserving cause category. Review generated samples for quality. Add approved synthetic samples to the training split only — never to validation or test.

PROJECT REQUIREMENTS DOCUMENT

EDS 6352 – Natural Language Processing | Fall 2026

Classifying Gas Transmission & Gathering Pipeline Incident Causes

Using BiGRU · BiLSTM+Attention · BERT-base

Team 1 | Sakina Saifuddin · Muhammad Murtaza · Parva Shah

Generated: September 30, 2026

5. Parva Shah — BiGRU Classifier

■ Parva Shah

BiGRU Text Classifier · FastText Pretrained Embeddings

5.1 Model Architecture

- **Embedding Layer:** Load pretrained FastText word vectors (300-dimensional, e.g., cc.en.300.bin). Build an embedding matrix mapping each word in the training vocabulary to its FastText vector. Freeze the embedding layer during training (do not update vectors).
- **BiGRU Layer 1:** Bidirectional Gated Recurrent Unit with hidden size 128 (64 per direction → 128 concatenated). Apply dropout ($p=0.3$) on output.
- **BiGRU Layer 2 (optional):** Second stacked BiGRU layer (hidden size 128). Apply dropout ($p=0.3$). Stacking improves capacity for longer sequences.
- **Pooling:** Apply mean pooling over the final hidden states of both directions to get a fixed-length 128-d sentence vector.
- **Classification Head:** Fully connected Linear(128 → 8). Outputs 8 raw logits — one per cause class.
- **Activation:** No softmax at output (use nn.CrossEntropyLoss which handles softmax internally).

5.2 Embeddings — FastText Setup

- Download the **FastText Common Crawl English vectors**: cc.en.300.bin (binary format, ~7 GB) or the text format crawl-300d-2M.vec (~2.2 GB).
- Use the fasttext Python library to load the binary model: `ft = fasttext.load_model('cc.en.300.bin')`.
- FastText handles out-of-vocabulary words via **character n-gram subword decomposition** — particularly useful for domain-specific pipeline/engineering terms.
- Build a vocabulary from the training set. For each word, call `ft.get_word_vector(word)`. Store as a NumPy matrix and load into `nn.Embedding`.
- Words not in the FastText vocab will still receive a subword-derived vector — no zero-vector fallback needed.

5.3 Training Hyperparameters

Hyperparameter	Value / Range	Notes
Optimizer	Adam	Standard choice for RNN classifiers
Learning Rate	1e-3 (tune: 5e-4 to 2e-3)	Use LR scheduler if loss plateaus
Batch Size	32 or 64	Fit GPU memory on Colab
Max Epochs	30	With early stopping

Early Stopping	Patience = 5 epochs	Monitor validation macro-F1
Dropout	0.3	Applied after each BiGRU layer
Max Seq Length	300 tokens	Pad shorter, truncate longer
Grad Clipping	max_norm = 5.0	Prevents exploding gradients in RNNs
Exp 1 Loss	nn.CrossEntropyLoss(weight=w)	w = inverse class frequency
Exp 2 Loss	nn.CrossEntropyLoss()	Standard, on augmented data

5.4 Parva's Deliverables

- **bigru_model.py** — PyTorch model class definition (BiGRU architecture).
- **bigru_train.py / bigru_train.ipynb** — Training loop for Experiment 1 and Experiment 2.
- **bigru_evaluate.py** — Evaluation script producing classification report and confusion matrix.
- **bigru_exp1_results.json** and **bigru_exp2_results.json** — Saved metric outputs.
- **bigru_best_model.pt** — Saved PyTorch checkpoint of the best validation model.
- Contribute equally to preprocessing pipeline, synthetic data generation, and final report.

6. Muhammad Murtaza — BiLSTM + Attention Classifier

■ Muhammad Murtaza

BiLSTM + Attention Text Classifier · FastText Pretrained Embeddings

6.1 Model Architecture

- **Embedding Layer:** Same FastText pretrained embedding matrix as BiGRU (shared vocab). Frozen during training.
- **BiLSTM Layer 1:** Bidirectional LSTM, hidden size 128 (64 per direction). Dropout (p=0.3) on output.
- **BiLSTM Layer 2:** Second stacked BiLSTM layer (hidden size 128). Dropout (p=0.3). Captures hierarchical sequential features.
- **Attention Mechanism:** Additive (Bahdanau-style) attention over the full sequence of BiLSTM hidden states. Computes a scalar attention weight α_t for each timestep t . The context vector is the attention-weighted sum: $\mathbf{c} = \sum \alpha_t \cdot \mathbf{h}_t$. This allows the model to focus on the most diagnostic words in the narrative.
- **Classification Head:** Linear(256 $\rightarrow$ 8) — input is the context vector. Outputs 8 raw logits.
- **Why Attention:** Narratives are long (~1,019 chars). Attention lets the model focus on key phrases like 'corrosion', 'backhoe strike', 'relief valve malfunction' regardless of their position.

6.2 Attention Implementation Notes

- Implement attention as a separate Attention PyTorch module for modularity and testability.
- Attention input: all hidden states from the final BiLSTM layer, shape (batch, seq_len, 256).
- Learnable attention weight vector: nn.Linear(256, 1, bias=False) $\rightarrow$ squeeze to (batch, seq_len) $\rightarrow$ softmax.
- Output context vector: weighted sum of hidden states, shape (batch, 256).
- Log or visualise attention weights on a few test examples to validate correctness (e.g., verify weights peak on cause-related words).

6.3 Training Hyperparameters

Hyperparameter	Value / Range	Notes
Optimizer	Adam	Consistent with BiGRU

Learning Rate	1e-3 (tune: 5e-4 to 2e-3)	Same search space as Parva for fair comparison
Batch Size	32 or 64	Fit GPU memory
Max Epochs	30	With early stopping
Early Stopping	Patience = 5 epochs	Monitor validation macro-F1
Dropout	0.3	After BiLSTM layers
Max Seq Length	300 tokens	Same as BiGRU
Grad Clipping	max_norm = 5.0	Prevents LSTM gradient issues
Exp 1 Loss	CrossEntropyLoss(weight=w)	Same class weights as Parva
Exp 2 Loss	CrossEntropyLoss()	On augmented data

6.4 Muhammad's Deliverables

- **bilstm_attention_model.py** — PyTorch model class (BiLSTM + Attention).
- **attention_module.py** — Standalone Attention module for reuse and testing.
- **bilstm_train.py / bilstm_train.ipynb** — Training loop for Experiment 1 and 2.
- **bilstm_evaluate.py** — Evaluation producing per-class metrics and attention heat-map visualisation.
- **bilstm_exp1_results.json** and **bilstm_exp2_results.json** — Saved metric outputs.
- **bilstm_best_model.pt** — Saved checkpoint of best validation model.
- Contribute equally to preprocessing pipeline, synthetic data generation, and final report.

7. Sakina Saifuddin — BERT-base-uncased + Classification Head

■ Sakina Saifuddin

BERT-base-uncased · Frozen Transformer · Classification Head

7.1 Architecture & Key Constraints

- **Base Model:** bert-base-uncased from HuggingFace Transformers — 12 transformer encoder layers, 768 hidden dimensions, 110M parameters.
- **Frozen Weights:** ALL pretrained transformer weights (attention, feedforward, LayerNorm) are FROZEN. Do NOT call optimizer on BERT parameters. Only the classification head parameters are trained.
- **BERT as Feature Extractor:** Pass the lowercased, tokenised narrative through BERT. Extract the **[CLS] token embedding** (first token, 768-d) as the sentence representation.
- **Classification Head:** Linear(768 → 8) — maps the [CLS] embedding to 8 class logits. Optionally add a dropout layer before the linear: Dropout(p=0.1) → Linear(768, 8).
- **BERT Embeddings:** BERT uses its own WordPiece tokeniser and positional/segment embeddings — do NOT use FastText with BERT.
- **Input Format:** Use BertTokenizer.encode_plus() to produce input_ids, attention_mask, and token_type_ids. Max length = 512 tokens.

7.2 Freezing BERT Weights — Implementation

Use the following pattern in PyTorch to freeze all BERT transformer parameters:

```
from transformers import BertModel
bert = BertModel.from_pretrained('bert-base-uncased')
for param in bert.parameters():
    param.requires_grad = False # Freeze all BERT weights
# Only the classification head (Linear layer) will be trained
```

- Verify that only the classification head parameters appear in the optimizer: print the count of trainable parameters — should be $768 \times 8 + 8 = 6,152$ parameters (vs 110M total).
- Validate by checking `sum(p.numel() for p in model.parameters() if p.requires_grad)` before training.
- Since transformer weights are frozen, BERT effectively acts as a static feature extractor — training is fast and does not require GPU for long.

7.3 Training Hyperparameters

Hyperparameter	Value / Range	Notes
Optimizer	AdamW (lr=2e-3)	Only optimises classification head
Learning Rate	1e-3 to 5e-3	Higher LR OK since only head is trained
Batch Size	16 or 32	BERT is memory-heavy; reduce if OOM
Max Epochs	20	Fewer epochs needed vs RNN models
Early Stopping	Patience = 5 epochs	Monitor validation macro-F1
Dropout	0.1 (before head)	BERT default dropout, applied to CLS
Max Seq Length	512 tokens	BERT hard maximum
Exp 1 Loss	CrossEntropyLoss(weight=w)	Same class weights as other models
Exp 2 Loss	CrossEntropyLoss()	On augmented training data
Library	HuggingFace transformers	pip install transformers

7.4 Sakina's Deliverables

- **bert_classifier_model.py** — PyTorch model class wrapping BertModel + classification head.
- **bert_train.py / bert_train.ipynb** — Training loop for Experiment 1 and 2 with frozen BERT.
- **bert_evaluate.py** — Evaluation producing per-class metrics.
- **bert_exp1_results.json** and **bert_exp2_results.json** — Saved metric outputs.
- **bert_best_model.pt** — Saved checkpoint of best validation model.
- Contribute equally to preprocessing pipeline, synthetic data generation, and final report.

PROJECT REQUIREMENTS DOCUMENT

EDS 6352 – Natural Language Processing | Fall 2026

Classifying Gas Transmission & Gathering Pipeline Incident Causes

Using BiGRU · BiLSTM+Attention · BERT-base

Team 1 | Sakina Saifuddin · Muhammad Murtaza · Parva Shah

Generated: September 30, 2026

8. Shared Responsibilities (All Members)

The following tasks are divided **equally** among all three team members. Each member must contribute meaningfully — coordination should happen via Git pull requests and a shared project board.

Phase	Task	Owner	Est. Effort
Preprocessing	Build the shared cleaning pipeline (Section 3): lowercasing, whitespace stripping, HTML removal. Run once and save the cleaned CSV for all members to use.	All 3 (lead rotates per step)	3–4 hrs
Synthetic Data	Generate AI-assisted paraphrased narratives for minority classes. Review and approve 10–15 samples per seed narrative per minority class. Save as synthetic_narratives.csv for Exp 2.	All 3 equally	3–4 hrs
Class Weights	Compute per-class weights (inverse frequency) once. Document the 8 weight values in a shared class_weights.json. All three models read from this file for Experiment 1.	All 3	1 hr
Split Indices	Run the stratified 70/15/15 split once with random_state=42. Save the row indices to split_indices.json. Every member must load from this file — never re-split independently.	All 3 agree on	1 hr
Evaluation	Agree on and implement a shared evaluate() function that accepts (y_true, y_pred) and returns the full classification_report + confusion matrix. All three members call the same function so results are directly comparable.	All 3	2 hrs
Results Handoff	Each member saves their results in the agreed JSON format (see Section 11). Share exp1_results.json and exp2_results.json per model when training is complete, so results can be compiled into the master comparison table later.	Each member	Ongoing

9. Evaluation Metrics

Every model, in every experiment, must report ALL of the following metrics on the **test set**. Use sklearn.metrics.classification_report and confusion_matrix.

Metric	Scope	Purpose
Accuracy	Overall	Fraction of correctly classified test instances
Precision (per class)	Per-class	TP / (TP + FP) for each of the 8 cause classes

Recall (per class)	Per-class	TP / (TP + FN) for each of the 8 cause classes
F1-score (per class)	Per-class	Harmonic mean of Precision and Recall per class
Macro-averaged Precision	Macro	Unweighted average of per-class Precision — treats all classes equally
Macro-averaged Recall	Macro	Unweighted average of per-class Recall
Macro-averaged F1	Macro	Primary comparison metric — macro-F1 is fairest for imbalanced classes
Micro-averaged Precision	Micro	Computed globally by counting TP/FP across all classes
Micro-averaged F1	Micro	Equivalent to accuracy for single-label classification
Confusion Matrix	Overall	8×8 matrix showing per-class prediction distributions

Comparison Structure: Once all three members complete training, results will be compiled into a master comparison table covering all 3 models x 2 experiments = 6 result sets. Macro-F1 is the primary ranking metric. The best-performing combination will inform the next phase of the project (deployment). For now, each member must produce and save all 10 metrics listed above for both experiments.

PROJECT REQUIREMENTS DOCUMENT

EDS 6352 – Natural Language Processing | Fall 2026

Classifying Gas Transmission & Gathering Pipeline Incident Causes

Using BiGRU · BiLSTM+Attention · BERT-base

Team 1 | Sakina Saifuddin · Muhammad Murtaza · Parva Shah

Generated: September 30, 2026

10. Technology Stack & Environment

The table below lists the recommended tools and libraries for this project. **This is a suggestion, not a strict mandate** — team members are free to use tools they are comfortable with. However, it is **strongly recommended to align on the mandatory items** (Python version, PyTorch, and output formats) since we will need to merge results and code later in the project. Keeping these consistent now will save significant effort during the merger phase.

Category	Tool / Library	Purpose / Notes	Status
Language	Python 3.10+	Primary development language. All members must use Python 3.10 or above to avoid compatibility issues.	Mandatory
Deep Learning	PyTorch 2.x	All model implementations (BiGRU, BiLSTM, BERT head). Use PyTorch so model checkpoints (.pt files) are compatible across members.	Mandatory
Transformers	HuggingFace transformers	BERT model and tokeniser (Sakina only). Other members do not need this.	Mandatory (Sakina)
Embeddings	FastText cc.en.300.bin	Pretrained 300-d word vectors (Parva + Muhammad). Use the same binary model file so embeddings are identical.	Mandatory (Parva & Muhammad)
Data / Eval	pandas, scikit-learn, NumPy	Data loading, splitting, and metrics. All members must use sklearn classification_report for consistent metric output.	Mandatory
Output Format	JSON for results	Save exp1_results.json and exp2_results.json in the agreed schema (see Section 11). Critical for merger.	Mandatory
Visualisation	matplotlib / seaborn	Confusion matrices, training loss curves, attention weight plots. Better if used — makes comparison easier during merger.	Suggested
Notebooks	Google Colab (GPU)	Model training with GPU acceleration. Better if used — ensures everyone has comparable compute resources.	Suggested
IDE	VS Code	Local development. Any IDE works, but VS Code is recommended for consistency in formatting.	Suggested
Version Control	Git + GitHub	Highly recommended — better if used, keeping in mind we need to merge our work later. Even just sharing a folder works for now.	Suggested

Legend:

Mandatory

= Required for all members. Deviating will create merger problems.

Suggested

= Not mandatory. Use if it helps you — better if the team aligns on these for easier merging later.

11. Ideal Individual File Structure (Per Member)

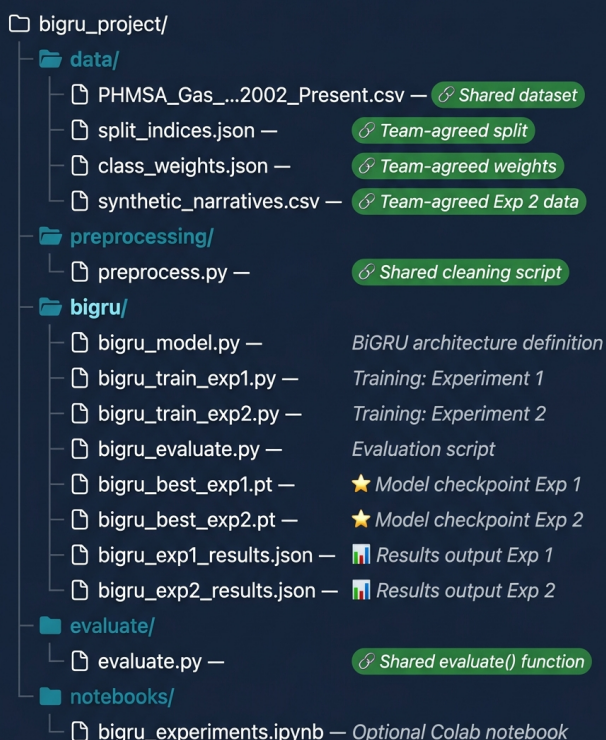
Why does this matter? Once everyone finishes training their model, we will merge all three members' work into one shared project folder. If every person follows the same folder layout and file-naming rules below, merging will take minutes instead of hours. The green-labelled files in each diagram are **shared team files** — every member must use the exact same copies of those files (the same dataset, the same split indices, the same class weights). Your model-specific files go inside your own named subfolder.

GREEN label = Shared file — use the exact team-agreed copy, do not re-create independently.

STAR (★) = Saved model checkpoint — this is what gets shared for the merger.

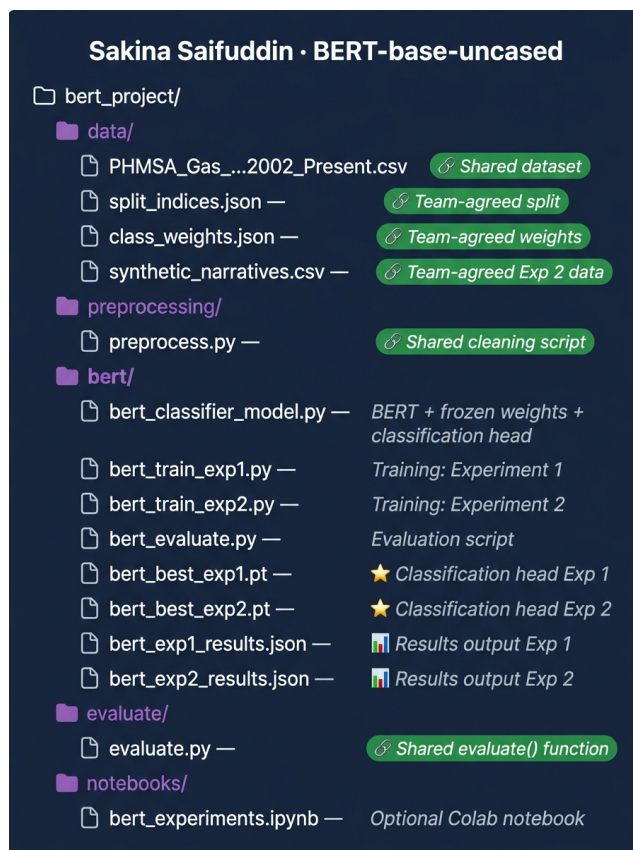
CHART (📊) = Results JSON — must follow the agreed schema in Section 11.4 below.

Parva Shah · BiGRU Classifier



Muhammad Murtaza · BiLSTM + Attention





11.4 Agreed Results JSON Schema — Critical for Merger

Every member must save their experiment results in **exactly** the JSON format shown below. Do not rename keys, do not change the structure, do not round values. When all three members share their JSON files, the comparison table can be generated automatically without any manual reformatting. Use **sklearn's classification_report(output_dict=True)** — its output maps directly to the per_class block. Save with **json.dump(results, f, indent=2)**.

Required Results JSON Schema — All Members Must Follow This Exactly

```
{
  "model": "BiGRU",
  "experiment": "exp1",
  "experiment_description": "...",
  "accuracy": 0.842,
  "macro_precision": 0.000,
  "macro_recall": 0.000,
  "macro_f1": 0.000,
  "micro_precision": 0.000,
  "micro_recall": 0.000,
  "micro_f1": 0.000,
  "per_class": {
    "EQUIPMENT FAILURE": { "precision": 0.0, "recall": 0.0,
                          "f1": 0.0, "support": 0 },
    "CORROSION FAILURE": { "precision": 0.0, "recall": 0.0,
                          "f1": 0.0, "support": 0 },
    "... (all 8 classes) ..."
  },
  "confusion_matrix": [...8x8 matrix...]
}
```

← Model: "BiGRU"
 ← Experiment: "exp1"
 ← Experiment_description: field
 ← Accuracy: 0.842
 ← 'Macro_precision': 0.000,
 ← Macro_recall": 0.000
 ← Primary primary ranking metric
 ← Micro_precision": 0.000,
 ← Micro_recall": 0.000,
 ← Micro_f1": 0.000,

Save with: `json.dump(results, f, indent=2)` — Do NOT round values

This PRD covers the individual model training phase of the EDS 6352 NLP Final Project (Fall 2026). Sections 10 and 11 (Tech Stack and File Structure) are designed specifically to ensure smooth merging of all three members' work in the next phase of the project. Merger, deployment, and report writing will be addressed in a separate document once all models are trained.